NGO Registration and Donation Management System

A PROJECT REPORT

ON

NSS-WEB-PROJECT

Web-Based NGO Management & Donation Platform

SUBMITTED BY:

Name: Amardeep Kumar, Chandan Kumar, Shashank Saha

Branch: Electrical and Electronic

Enrollment No:23115011,2311607,23115133

Institute: IIT Roorkee

SUBMITTED TO:

NSS IIT Roorkee

(Technical Team)

Session: 2025-2026

ABSTRACT

In the digital era, Non-Governmental Organizations (NGOs) increasingly rely on online platforms to manage supporter registrations and collect donations efficiently. However, many existing systems tightly couple user registration with payment completion, resulting in loss of valuable user data when transactions fail or are abandoned. This limitation reduces future engagement opportunities and affects transparency and accountability.

This project presents the design and implementation of an **NGO Registration and Donation Management System** that separates the registration workflow from the donation process. The system ensures that user information is securely stored regardless of payment outcome, while every donation attempt is accurately tracked with clear status indicators such as success, pending, and failure. Role-based authentication enables secure access for both users and administrators. Administrators can monitor registrations, analyze donation trends, and maintain audit records through a centralized dashboard.

A sandbox-based payment simulation is used to demonstrate ethical transaction handling without real financial risk. The system emphasizes data integrity, security, scalability, and modular architecture, making it suitable for both academic evaluation and real-world deployment. This solution enhances transparency, operational efficiency, and long-term engagement for NGOs conducting online fundraising campaigns.

INTRODUCTION

1. Introduction

Non-Governmental Organizations (NGOs) increasingly rely on digital platforms to manage registrations and collect donations efficiently. In many traditional systems, user data is lost when a payment fails or is not completed, reducing transparency and future engagement opportunities.

The **NGO Registration and Donation Management System** address this issue by separating user registration from the donation process. It ensures that registration data is securely stored regardless of payment outcome, while all donation attempts are accurately tracked. The system provides role-based access for administrators to monitor registrations and donations, ensuring transparency, data integrity, and ethical handling of transactions.

1.2 Problem Statement

Many NGO online platforms tightly couple user registration with the donation process. When a user abandons a transaction or a payment fails due to technical issues, the system often loses

the user's registration data. This results in loss of potential supporters, inaccurate records, and reduced transparency for administrators. Additionally, administrators lack effective tools to monitor registrations, track donation statuses, and generate reliable reports. There is a need for a secure and reliable system that separates registration from donation, preserves user data regardless of payment outcome, and provides accurate monitoring and ethical handling of transactions.

1.3 Objectives

The main objectives of the NGO Registration and Donation Management System are:

- To design a secure system that allows users to register independently of the donation process.
- To ensure that user data is stored reliably even if a payment fails or is not completed.
- To enable users to optionally donate any amount with transparent status tracking.
- To accurately record and display donation statuses as success, pending, or failed.
- To provide administrators with real-time visibility into user registrations and donation activities.
- To maintain ethical handling of payment transactions without false success reporting.
- To improve transparency, data integrity, and operational efficiency for NGO management.

SYSTEM ARCHITECTURE

2. System Architecture

The application follows a **Model-View-Controller (MVC)** architecture, ensuring a clean separation of concerns between the user interface, business logic, and data storage.

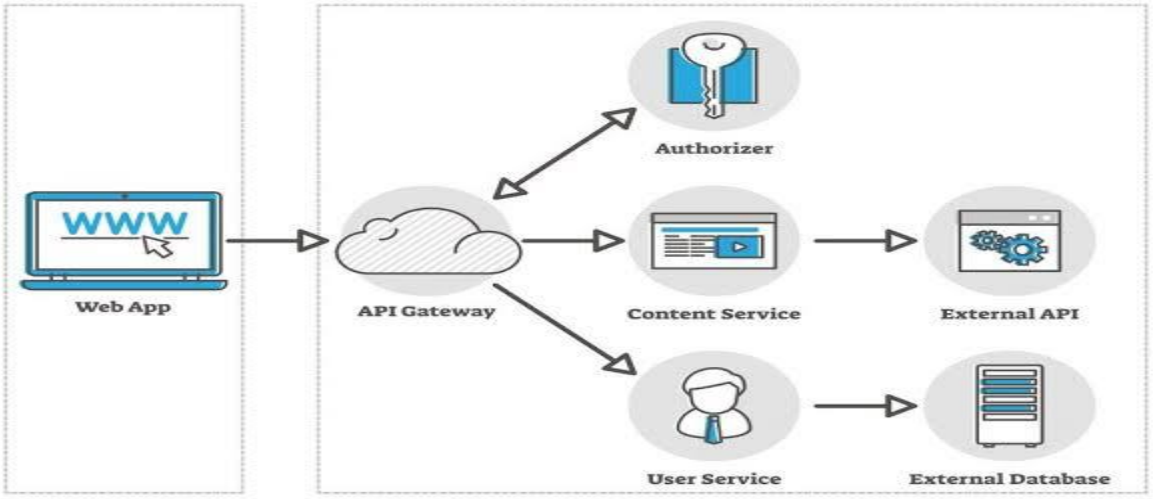
Client (Frontend): The user interacts with the React.js interface. Data is sent via Axios to the backend API.

Server (Backend): The Node.js/Express server validates the request.

Database: User details are saved to MongoDB Atlas immediately.

Payment Gateway: An order ID is generated via Razor pay. The user attempts payment.

Verification: The payment signature is verified on the server. If valid, the status is updated to "Success".



2.2 Data Flow Diagram:

- **Step 1:** User enters Email -> System sends OTP via Node mailer.
- **Step 2:** User verifies OTP -> System creates User Profile in MongoDB.
- **Step 3:** User clicks Donate -> System calls Razor pay API -> Payment Window opens.
- **Step 4:** Transaction Complete -> Webhook/Callback updates Donation Collection.

TECHNOLOGY STACK

The NGO Registration and Donation Management System is developed using modern web technologies to ensure scalability, security, and ease of maintenance. The selected technology stack supports rapid development, reliable data storage, and secure communication. It is built using the **MERN Stack**, chosen for its scalability, JSON-native nature, and unified JavaScript development environment.

3. Frontend

- **React.js (Vite):** Used for building a dynamic Single Page Application (SPA). Vite ensures fast build times.
- **Tailwind CSS:** A utility-first CSS framework used for responsive styling and modern UI design.
- **Framer Motion:** Used to create smooth animations for the multi-step Volunteer Registration form.
- **Axios:** Handles HTTP requests (GET, POST) to communicate with the backend.

3.1 Backend

- **Node.js:** The runtime environment for executing JavaScript on the server.
- **Express.js:** A minimal web framework used to create RESTful API endpoints (/api/auth, /api/payment).
- **Nodemailer:** Configured with Google SMTP (App Password) to send secure One-Time Passwords (OTPs).
- **Crypto:** A built-in Node.js module used to verify the HMAC SHA256 signature of Razorpay transactions.

3.2 Database & External API

- **MongoDB Atlas:** A cloud-based NoSQL database used to store User, Volunteer, and Donation records.
- **Razorpay (Test Mode):** The payment gateway API used to simulate financial transactions securely.

FUNCTIONAL MODULES

The NGO Registration and Donation Management System is divided into multiple functional modules to ensure modularity, scalability, and ease of maintenance. Each module performs a specific role in the overall system operation.

4. Authentication Module

- **OTP-Based Login:** Eliminates the need for passwords. Users receive a 6-digit code on their email.
- **Role-Based Access Control (RBAC):** The system distinguishes between 'User' and 'Admin'.

- *Admins* are redirected to the Dashboard.
- *Users* are redirected to the Donation/Profile page.

4.1 User Management Module

- **Registration:** Users can sign up with basic details (Name, Email, Phone, Interest).
- **Cause Selection:** Volunteers can choose specific domains (Education, Health, Environment).
- **Donation History:** Users can view a personal log of all their contributions and their current status.

4.2 Admin Dashboard Module

- **Overview Stats:** Displays Total Users, Total Volunteers, and Total Funds Collected.
- **User Management:** A tabular view of all registered users with search and filter capabilities.
- **Payment Audit:** Administrators can view detailed logs of every transaction attempt, helping identify technical failures vs. user cancellations.

DATABASE DESIGN

5. Database Schema

The database is designed to ensure data integrity, scalability, and efficient retrieval of information. The system uses a NoSQL database (MongoDB) with well-defined collections to store user information and donation records separately. This separation ensures that user data remains persistent even if a donation fails or is not completed. We use three primary collections in MongoDB.

5.1 User Collection

This collection stores all registered user and administrator details.

Fieldname	Data Type	Description
_id	Object Id	Unique user identifier

name	String	User full name
email	String	Unique email address
password	String	Encrypted password
role	String	User role (user / admin)
otp	String	One-time password (temporary)
otpExpires	Date	OTP expiration time
isVerified	Boolean	Verification status
createdAt	Date	Account creation timestamp

5.2 Donation Collection

This collection stores all donation transaction records.

Fieldname	Data Type	Description
_id	Object Id	Unique donation ID
User Id	Object Id	Reference to User collection
amount	Number	Donation amount
status	String	Payment status (Success / Pending / Failed)
Transaction Id	String	Payment transaction reference
timestamp	Date	Transaction date and time

5.3 Volunteer Collection

Stores volunteer participation and cause selection details.

Fieldname	Data Type	Description
_id	ObjectId	Unique volunteer record ID
userId	ObjectId	Reference to User collection
cause	String	Selected cause (Education, Health, Environment, etc.)
availability	String	Time availability (Weekdays, Weekends, Flexible)
skills	String	Relevant skills or interests
motivation	String	Reason for volunteering
Joined At	Date	Volunteer registration date

INPLEMENTATION DETAILS

This section explains the core implementation logic, technical decisions, and how system requirements are achieved in practice.

6. Decoupling Registration from Donation

Challenge: In standard flows, user data is often sent only *after* payment success.

Solution: We split the API call. First, a POST /Api/register call saves the user to MongoDB.Only after the server confirms the user is saved does the frontend trigger the Payment Modal. This ensures 100% data capture.

6.1 Secure Payment Verification

Challenge: Frontend payment success messages can be spoofed (hacked).

Solution: The system does not trust the frontend. When Razor pay returns a success flag, the backend performs a **Cryptographic Verification.**

6.2 Email Delivery Security

Challenge: Google blocks standard login for sending emails via code.

Solution: We implemented **2-Step Verification** and generated a 16 digit **App Password** for the Gmail SMTP configuration, ensuring reliable OTP delivery without exposing the primary account password.

PROJECT SCREENSHOTS

Fig 7.1: The Landing Page

(Paste screenshot of Home Page)

Fig 7.2: Volunteer Registration (Step-by-Step Form)

(Paste screenshot of the Form)

Fig 7.3: Razorpay Payment Integration (Test Mode)

(Paste screenshot of the Razorpay Popup window)

Fig 7.4: Admin Dashboard

(Paste screenshot of the Admin Panel showing charts or tables)

CONCLUSION

The NGO Registration and Donation Management System successfully addresses the limitations of traditional donation platforms by separating user registration from the payment process. This ensures that valuable user data is preserved even when a transaction fails or is abandoned, improving transparency and long-term engagement opportunities for NGOs.

The system provides secure authentication, accurate donation tracking, and role-based administrative control. All donation attempts are recorded with clear status indicators, enabling reliable auditing and reporting. The use of a sandbox payment gateway ensures ethical handling of transactions while allowing safe testing and demonstration.

With a modular architecture and scalable design, the platform supports future enhancements and real-world deployment. Overall, the project demonstrates an effective digital solution that improves operational efficiency, data integrity, and accountability in NGO management systems.

FUTURE SCOPE

The NGO Registration and Donation Management System provides a strong foundation for digital NGO operations. Several enhancements can be implemented in the future to improve scalability, usability, and functionality:

- **Real Payment Gateway Integration:**
  Integrate live payment gateways such as Razorpay, Stripe, or PayPal for real-world transactions.

- **Automated Receipts and Tax Certificates:**
  Generate downloadable donation receipts and tax exemption certificates automatically.
- **Mobile Application Support:**
  Develop Android and iOS applications for wider accessibility.
- **Advanced Analytics Dashboard:**
  Provide graphical insights on donation trends, donor behavior, and campaign performance.
- **Multi-Language Support:**
  Enable support for multiple languages to reach a broader audience.
- **Notification System:**
  Implement email and SMS alerts for successful donations and important updates.
- **Volunteer Event Management:**
  Allow volunteers to register for events, track attendance, and receive notifications.
- **AI-Based Chat Support:**
  Integrate chatbots to assist users with queries and guidance.

REFERENCES

1. **React Documentation:** hflps://react.dev/
2. **Razorpay Developer Docs:** hflps://razorpay.com/docs/
3. **MongoDB Atlas Manual:** hflps://www.mongodb.com/docs/atlas/
4. **Nodemailer Documentation:** hflps://nodemailer.com/